

GPU / cuTile · Projekt B

Auto-Tuning für Tensor-Kontraktionen

Aus Einsum-String + Shapes automatisch eine gute cuTile-Tiling-Config finden

Niklas Becker-Klöser · Daria Elagina

cmk,ckn→cmn · acspx,bspy→abcyx · NVIDIA GB10 (DGX Spark)

Von Hand getunt skaliert nicht

- ▶ A05 & A06: L2-optimale Aufteilung von Hand hergeleitet.
- ▶ Für jede neue Kontraktion / Shape / GPU: neu nachdenken.
- ▶ Ziel: Eingabe = Einsum + Shapes → Ausgabe = gute Config.

Scope:

- ▶ GEMM (plain batched Matrixmultiplikation) - A05
- ▶ Ring (Tensor-Ring-Dekomposition) - A06

Kein allgemeiner Tuner!

2 Testfälle

A05 · batched Matmul

cmk, ckn → cmn

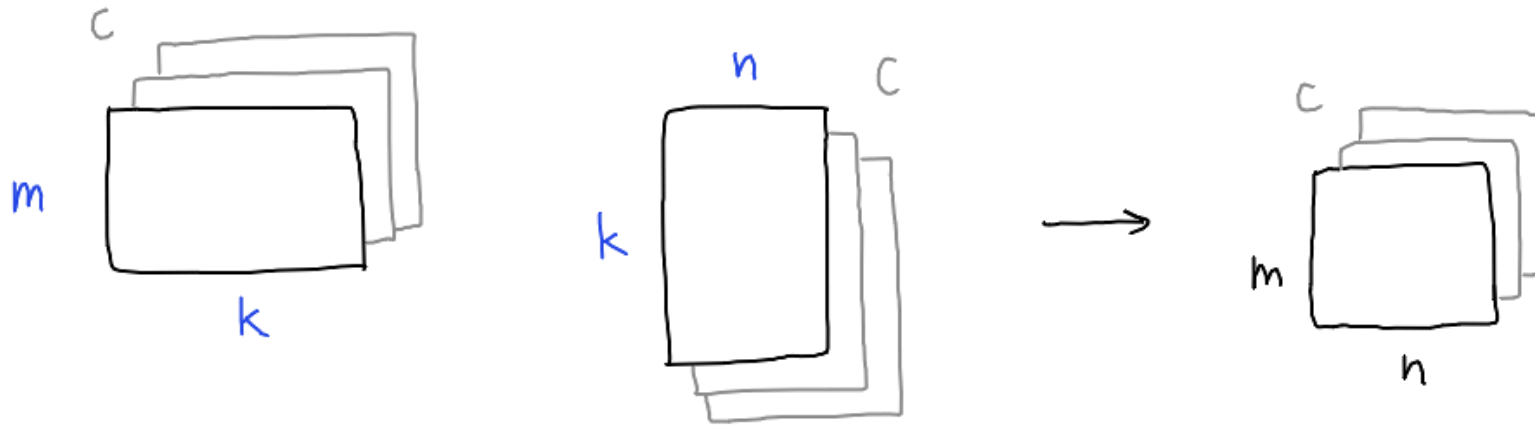
A06 · Tensor-Ring

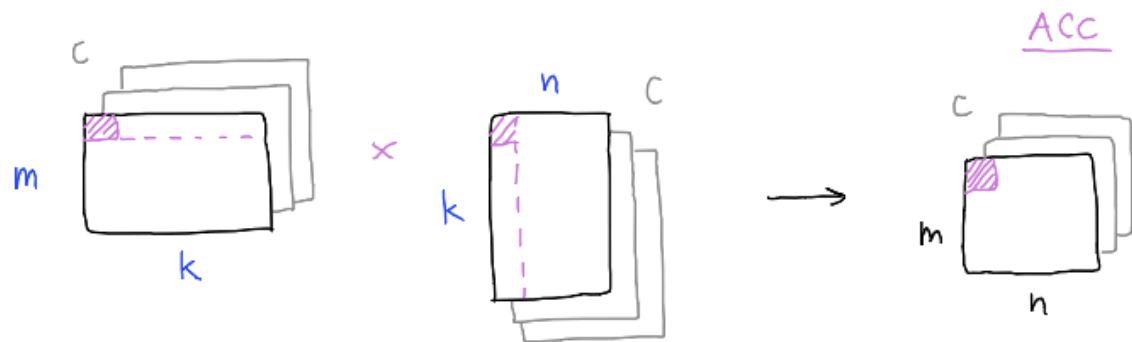
acsp_x, bsp_y → abcy_x

Die Performance steckt in der Config, nicht im Kernel-Code — genau das automatisieren wir!

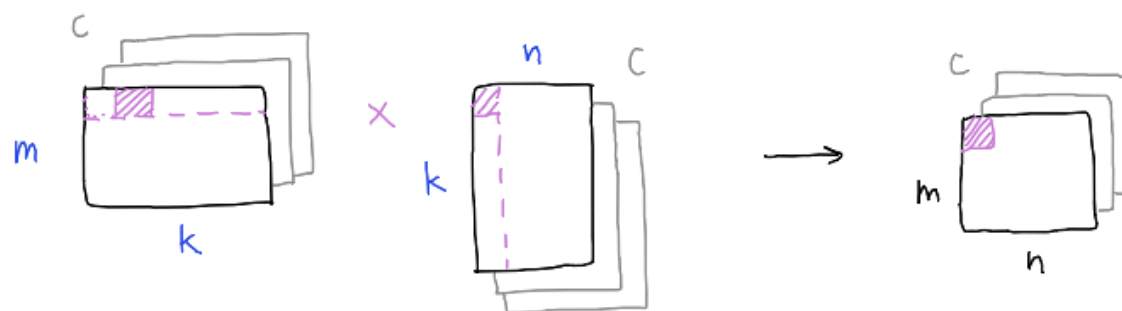
Einsum & Suchraum

$cmk, ckn \rightarrow cmn$

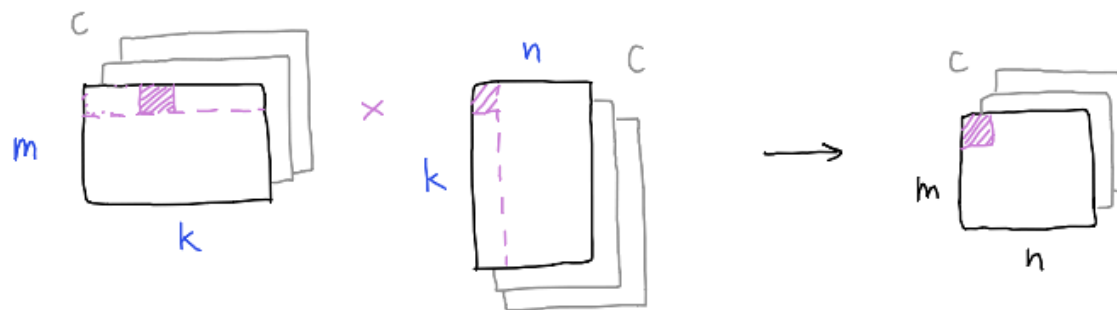




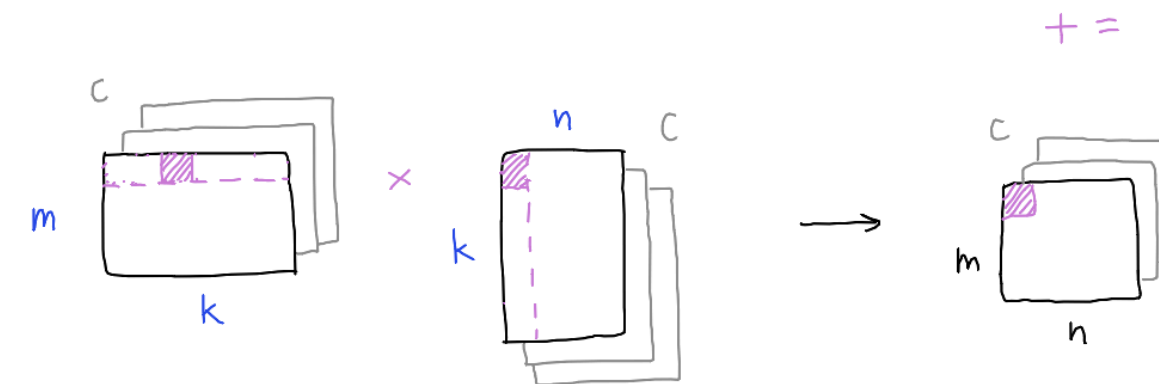
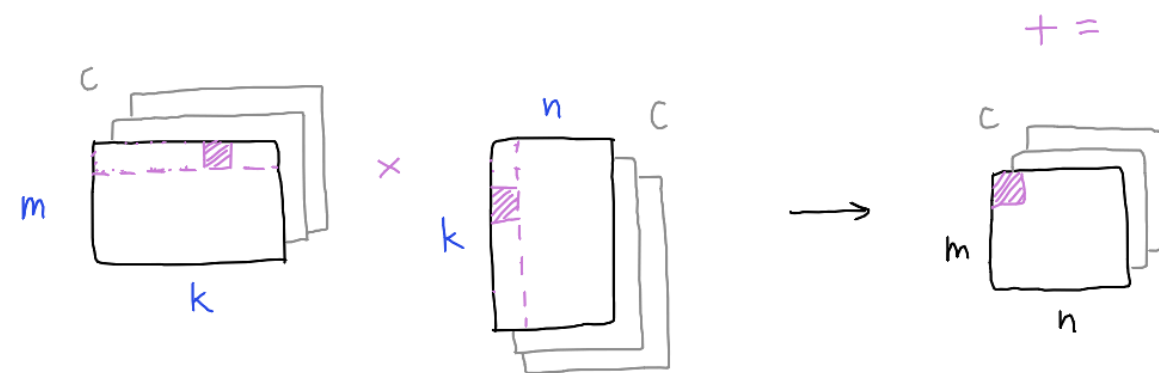
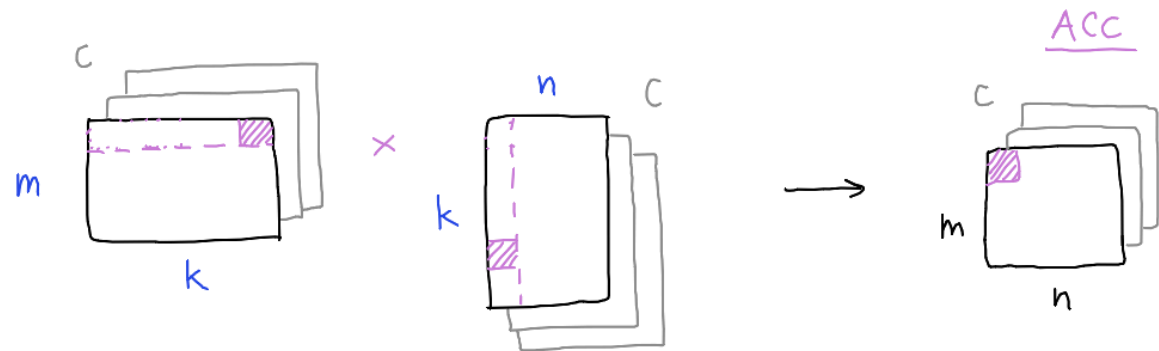
$+=$



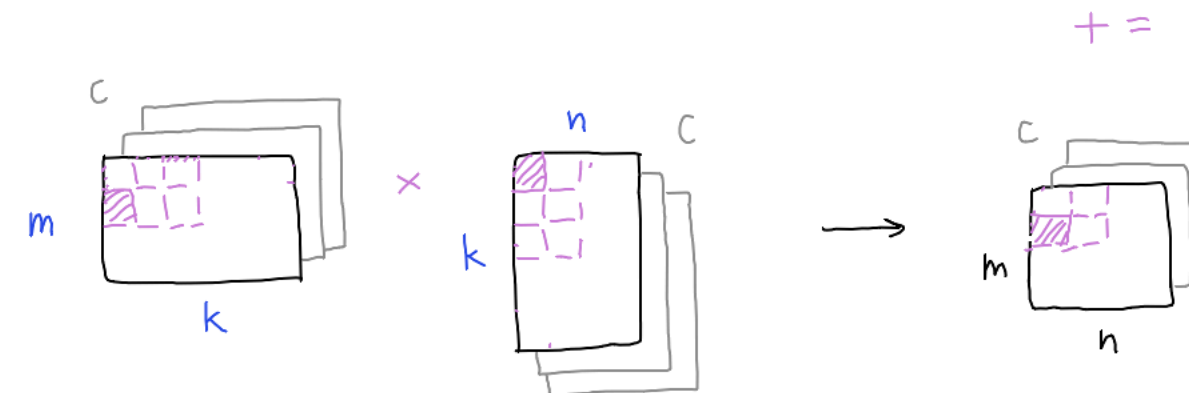
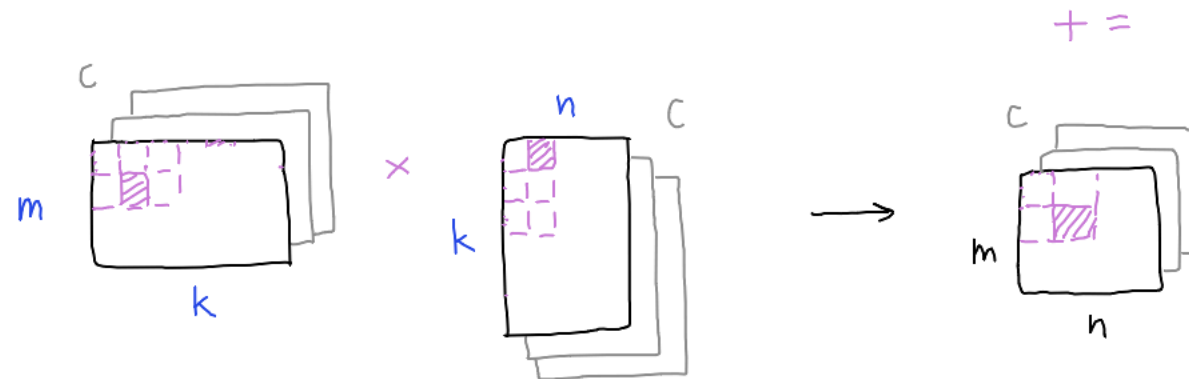
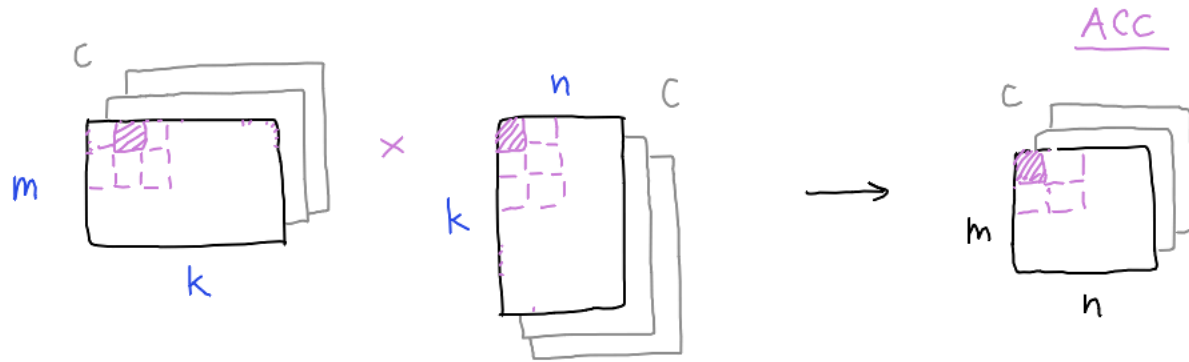
$+=$



USW...



USW...



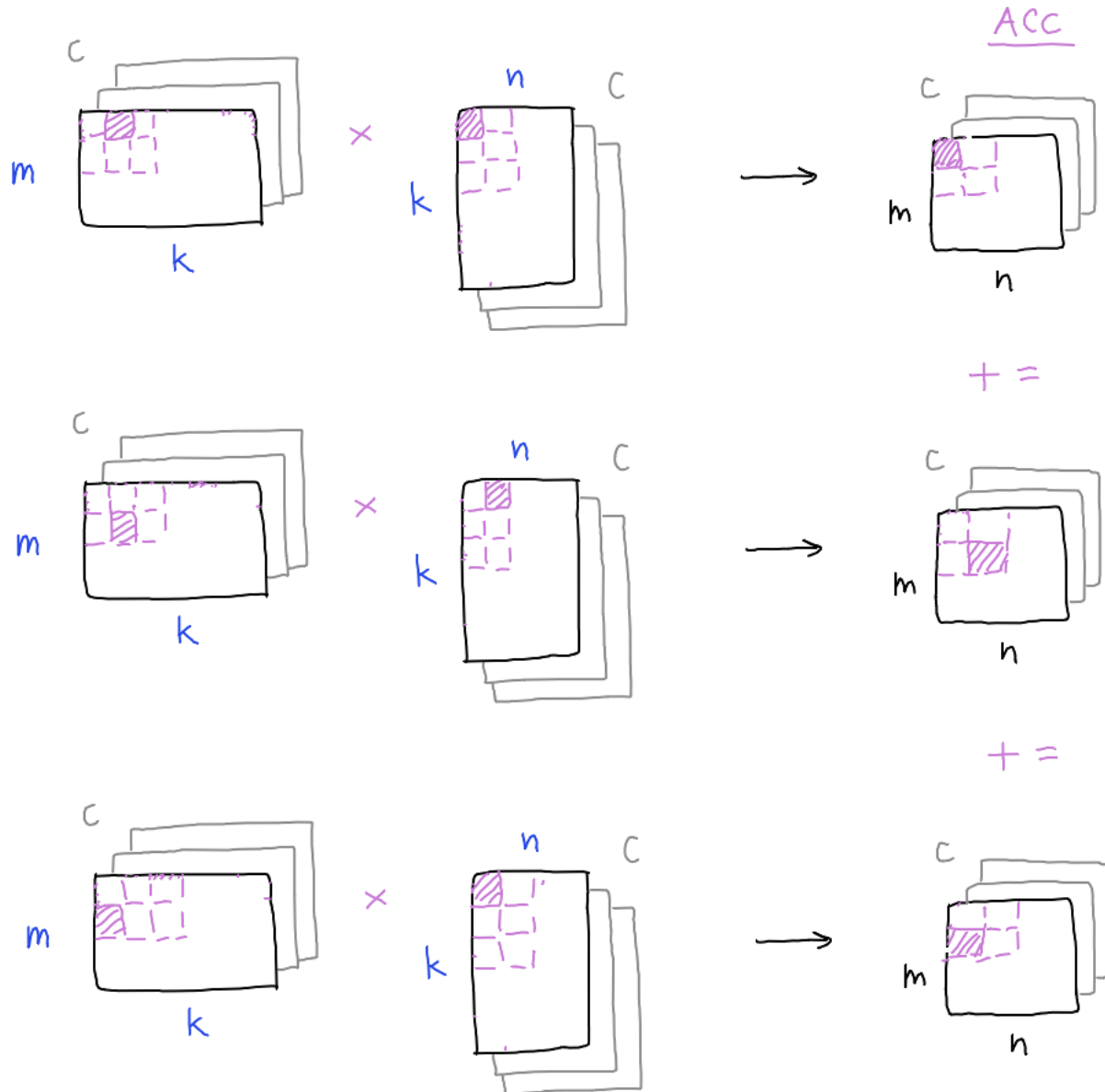
Mit:

A-Tile: 2x3 (m=2, k=3)

B-Tile: 3x2 (k=3, n=2)

Accumulator: 2x2 (m×n)

USW...



einzige Einschränkung:

Alle Beiträge, die zum selben Output-Element (c, m, n) gehören, müssen in denselben Akkumulator summiert werden

— relative Reihenfolge ist egal (*wegen Kommutativität und Assoziativität*)

Jede Multiplikation $A[c, m, k] \times B[c, k, n]$ muss genau einmal ausgeführt werden

—> rein kombinatorisch sehr viele Möglichkeiten!

Was grenzt den Suchraum? (*Performance Anspruch*)

- ▶ Tiles müssen Vielfache von 16 sein
- ▶ $M_PRIM * N_PRIM$ Accumulator soll in SM Register passen
- ▶ A-Tile & B-Tile müssen in Shared Memory eines SMs passen (+double buffering)

Einsum —> WAS?

Config —> WIE?

Die Performance steckt in der Config, nicht im Kernel-Code — genau das automatisieren wir!

TEIL 1

Umsetzung & Architektur

Pipeline · Suchraum · Kernel · Pruning · A06-Transfer

Was ist eine Tiling-Config?

Dim-Types: Rolle in der Matmul (m, n, k, c)

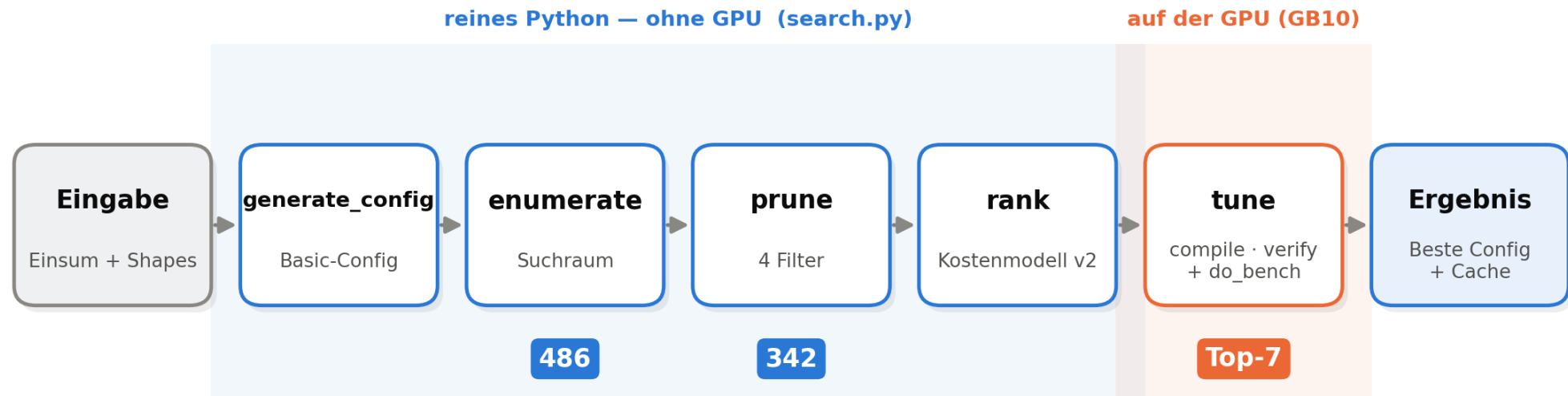
Dim-Size + Strides: Größe und Speicherlayout pro Tensor

Exec-Types: wie die Dimension ausgeführt wird

- PAR: auf die Grid-Dimension abgebildet, paralleles Starten der Blöcke, kein Loop
- SEQ: for-Schleife im Kernel
- PRIM: mma-Kachel; $M_PRIM / N_PRIM / K_PRIM$

Die Tuner-Pipeline

Die Tuner-Pipeline: von Einsum zur gemessen besten Config



nur die Top-7 werden auf der GPU gemessen.

Den Suchraum eingrenzen

- ▶ $M_PRIM, N_PRIM \in \{64, 128, 256\}$
- ▶ $K_PRIM \in \{32, 64, 128\}$
- ▶ $M_L2, N_L2 \in \{2, 4, 8\}$
- ▶ Variante $\in \{A, B\}$

$$3 \cdot 3 \cdot 3 \cdot 3 \cdot 3 \cdot 2 = 486$$

Kandidaten (nicht die „81“ aus dem Pitch)

✓ Akzeptanztest

Die handoptimierte A05-Config ist im Set — sonst könnte der Tuner sie nie finden.

enumerate.py: bewusst klein & hardware-freundlich gehalten; krumme Shapes werden hochgepadet (PaddingMode.ZERO).

Den Suchraum eingrenzen (*Herleitungsbeispiel*)

- ▶ $M_PRIM, N_PRIM \in \{64, 128, 256\}$

Warum ist 256 MAX?

1.4.1.1. Occupancy

The maximum number of concurrent warps per SM is 64 for compute capability 10.0 and 48 for compute capability 12.0. Other **factors influencing warp occupancy** [↗](#) are:

- The register file size is 64K 32-bit registers per SM.

Quelle: <https://docs.nvidia.com/cuda/blackwell-tuning-guide/index.html>

Accumulator:

- ▶ $M_PRIM * N_PRIM \rightarrow 256 * 256 = 65.536$
- ▶ Accumulator-Tile in den Registern von SM halten

Pruning: statische Filter, ohne Compile

1 MMA-Teilbarkeit

$$m_{\text{prim}}, n_{\text{prim}}, k_{\text{prim}} \bmod 16 = 0$$

2 SMEM-Budget

$$(m_{\text{prim}} \cdot k_{\text{prim}} + k_{\text{prim}} \cdot n_{\text{prim}}) \cdot 2 \text{ Byte} \cdot 2 \text{ Stages} \leq 98 \text{ KiB}$$

fp16 (2 Byte), double-buffering (2 stages)

3 Akku-Register

$$m_{\text{prim}} \cdot n_{\text{prim}} \leq \frac{1}{2} \cdot 65536$$

fp32-Akku liegt in Registern

4 Padding-Verschwendung

$$V_{\text{padded}} / V_{\text{orig}} \leq 8$$

Faktor 8.0 (maximal)

$$\frac{\text{padded}_m * \text{padded}_n * \text{padded}_k}{\text{orig}_m * \text{orig}_n * \text{orig}_k}$$

Ranking: Kostenmodell

Idealfall — alles passt perfekt ins Cache (jede Matrix 1×):

$$\text{cold} = M \cdot K + K \cdot N + M \cdot N \quad (\times 2 \text{ Byte})$$

Worst-Case-Traffic (kein L2-Reuse zwischen Gruppen):

$$A: M \cdot K \cdot \text{ceil}(N / (n_{l2} \cdot n_{\text{prim}})) \quad \leftarrow A \text{ pro } N\text{-Gruppe neu}$$

$$B: K \cdot N \cdot \text{ceil}(M / (m_{l2} \cdot m_{\text{prim}})) \quad \leftarrow B \text{ pro } M\text{-Gruppe neu}$$

$$C: M \cdot N$$

$$\text{worst} = A + B + C$$

passt eine Gruppe in den L2?

$$(m_{l2} \cdot m_{\text{prim}} + n_{l2} \cdot n_{\text{prim}}) \cdot K \cdot 2B \leq L2 \rightarrow \text{cold}$$

$$\text{sonst} \rightarrow \text{worst}$$

Roofline: $t = \max(t_{\text{mem}}, t_{\text{compute}})$

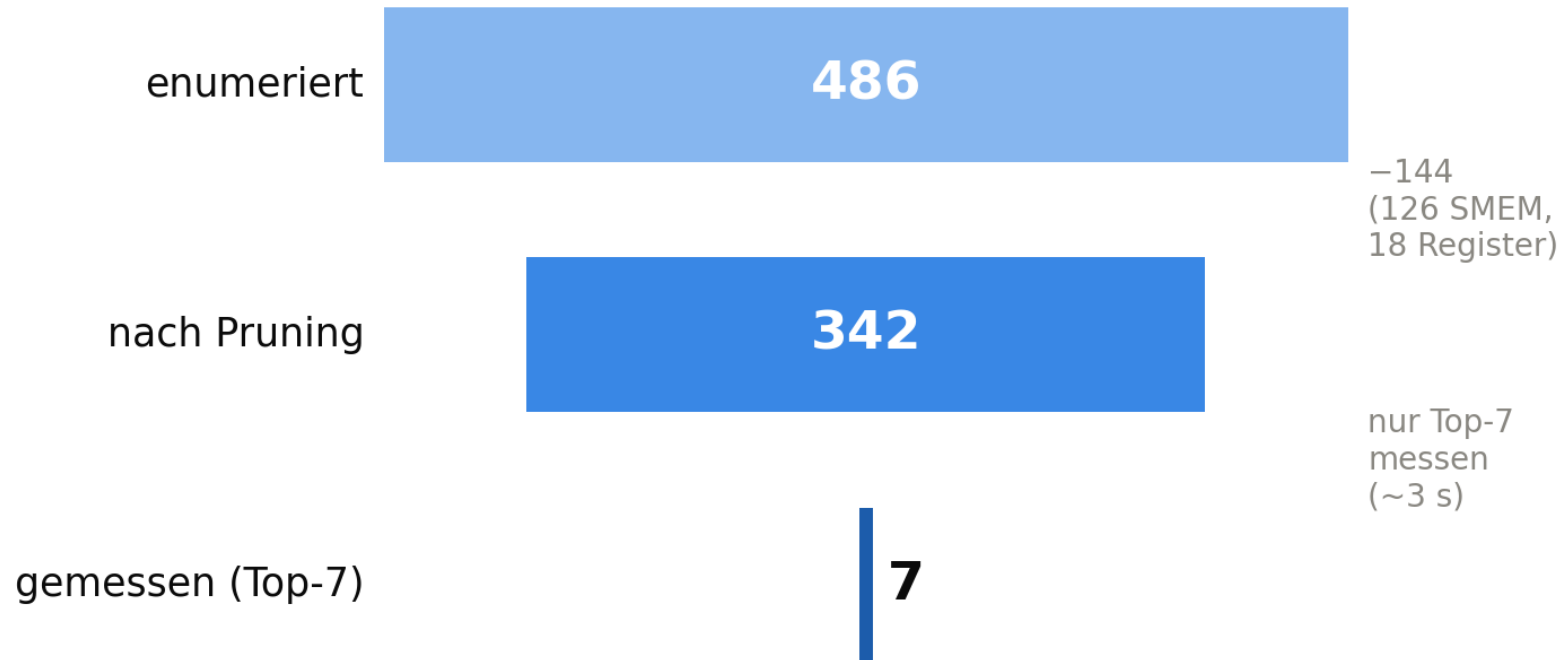
$$t_{\text{mem}} := (\text{Traffic in Bytes}) / \text{Peak_DRAM_Bandwidth} * 1000 \quad \# \text{ in ms}$$

$$t_{\text{compute}} = 2 \cdot M \cdot N \cdot K / (\text{Peak} \cdot \text{util})$$

util (0.0 bis 1.0) = wie gut man die Tensor Cores wirklich auslastet

Pruning: A05

Suchraum eingrenzen: 486 → 342 → 7 (A05)



Ein generischer Kernel

- ▶ Variante A: Swizzle über die Block-ID.
- ▶ Korrektheit gegen torch.einsum
(allclose)

Compile ~0.4 s pro Config,
auf der GB10 verifiziert —
der ganze Ansatz hängt an
dieser Spezialisierung.

```
@ct.kernel
def matmul_variant_a(A, B, C,
                     M_PRIM: ct.Constant[int],
                     N_PRIM: ct.Constant[int],
                     K_PRIM: ct.Constant[int],
                     M_L2: ct.Constant[int],
                     N_L2: ct.Constant[int],
                     num_m_l2_outer: ct.Constant[int],
                     num_n_l2_outer: ct.Constant[int],
                     num_k_outer: ct.Constant[int]):

    pid = ct.bid(0)
    n_l2_idx = pid % N_L2
    pid = pid // N_L2
    m_l2_idx = pid % M_L2
    pid = pid // M_L2
    n_l2_outer_idx = pid % num_n_l2_outer
    pid = pid // num_n_l2_outer
    m_l2_outer_idx = pid % num_m_l2_outer
    pid = pid // num_m_l2_outer
    c_idx = pid

    m_block = m_l2_outer_idx * M_L2 + m_l2_idx
    n_block = n_l2_outer_idx * N_L2 + n_l2_idx

    acc = ct.zeros((M_PRIM, N_PRIM), dtype=ct.float32)
    for k_it in range(num_k_outer):
        a_tile = ct.load(A, index=(c_idx, m_block, k_it),
                          shape=(1, M_PRIM, K_PRIM), padding_mode=ct.PaddingMode.ZERO)
        b_tile = ct.load(B, index=(c_idx, k_it, n_block),
                          shape=(1, K_PRIM, N_PRIM), padding_mode=ct.PaddingMode.ZERO)
        a_tile = ct.reshape(a_tile, (M_PRIM, K_PRIM))
        b_tile = ct.reshape(b_tile, (K_PRIM, N_PRIM))
        acc = ct.mma(a_tile, b_tile, acc)

    out = ct.reshape(acc, (1, M_PRIM, N_PRIM)).astype(ct.float16)
    ct.store(C, index=(c_idx, m_block, n_block), tile=out)
```

A06: eine zweite Struktur-Familie

```

@ct.kernel
def matmul_ring_a(A, B, C, M_PRIM: ct.Constant[int], ...,
                  SIZE_B: ct.Constant[int], SIZE_C: ct.Constant[int],
                  SIZE_S: ct.Constant[int], num_k_outer: ct.Constant[int]):
    pid = ct.bid(0) # a,c,b: extra M/N-Dims -> PAR
    b_idx = pid % SIZE_B; pid //= SIZE_B # a,c nur in A, b nur in B
    c_idx = pid % SIZE_C; pid //= SIZE_C # (kein geteilter Batch)
    a_idx = pid
    acc = ct.zeros((M_PRIM, N_PRIM), ct.float32)
    for s_it in range(SIZE_S): # 2. Reduktion s als SEQ-Loop
        for k_it in range(num_k_outer): # p als prim_k im mma
            tA = ct.load(A, index=(a_idx, c_idx, s_it, k_it, ...))
            tA = ct.permute(tA, (1, 0)) # Layout ist nicht mma-fertig
            tB = ct.load(B, index=(b_idx, s_it, k_it, ...))
            acc = ct.mma(tA, tB, acc)
    ct.store(C, index=(a_idx, b_idx, c_idx, y_block, x_block), ...)

```

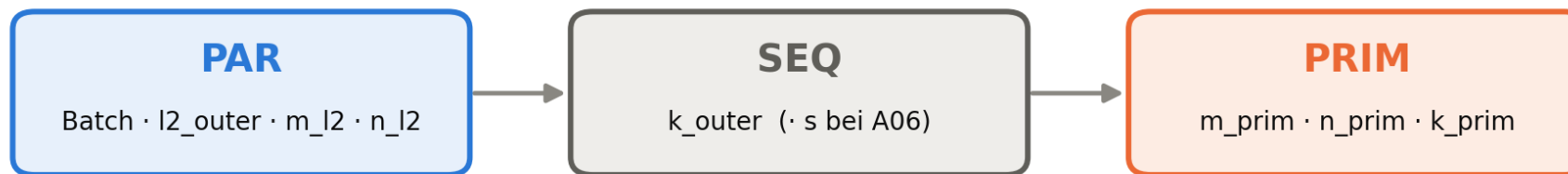
acsp_x, bsp_y -> abcy_x

- ▶ Nicht „doppeltes K“, sondern die Batch-Topologie, also wie die unabhängigen Achsen auf A und B liegen.
- ▶ A05: geteilter Batch c (in A, B, Out) A06: kein geteilter batch -> a, c nur in A; b nur in B.
- ▶ diese extra M/N-Dims laufen als parallele Blöcke —> zweiter Kernel Typ —> kein Umbau
- ▶ ▶ Der Tuner sucht Configs, nicht Kernel — A05-Pfad bleibt bitgleich.

Wie cuBLAS/CUTLASS (Template-Menge) oder Triton (autotune pro @jit): neue Topologie = neues Template, Configs getunt.

Wie wir die Reihenfolge der Dimensionen definiert haben

1 · Reihenfolge in der Config (verify):



K nie PAR · PRIM ganz rechts = je $\geq 1 \times M, N$ und K (die mma-Kachel)

2 · Split je Dimension (der Tuner variiert nur die Größen):

M → m_l2_outer · m_l2 · m_prim

N → n_l2_outer · n_l2 · n_prim

K → k_outer · – · k_prim

Farbe = Exec-Typ (wie oben):

blau = PAR (l2_outer · m_l2 · n_l2)

grau = SEQ (k_outer)

orange = PRIM (m/n/k_prim)

Variante A: m_l2/n_l2 = PAR (Swizzle)

Variante B: m_l2/n_l2 = SEQ-Loop

Jede Config ist by construction gültig: die Reihenfolge PAR | SEQ | PRIM steht fest (verify), der Tuner variiert nur die Split-Größen und A/B.

TEIL 2

Evaluation & Ergebnisse

Gegen Handkernel · gegen cuBLAS · über Shapes & GPUs

Benchmark-Setup: acht Shape-Regime je Familie

Acht Shape-Regime · einsum cmk, ckn → cmn

Regime	C	M	N	K	testet
square · b4	4	4096	4096	4096	Referenz (Heimvorteil)
square · b1	1	4096	4096	4096	ohne Batch
tall M≫N	1	8192	1024	4096	rechteckig, viele Zeilen
wide N≫M	1	1024	8192	4096	rechteckig, viele Spalten
small_k	1	4096	4096	512	kleines K → bandbreiten-nah
large_k	1	1024	1024	8192	großes K → compute-nah
krumm	2	1500	3000	1000	unteilbar → Padding-Pfad
batch16	16	1024	1024	1024	viele kleine Batches

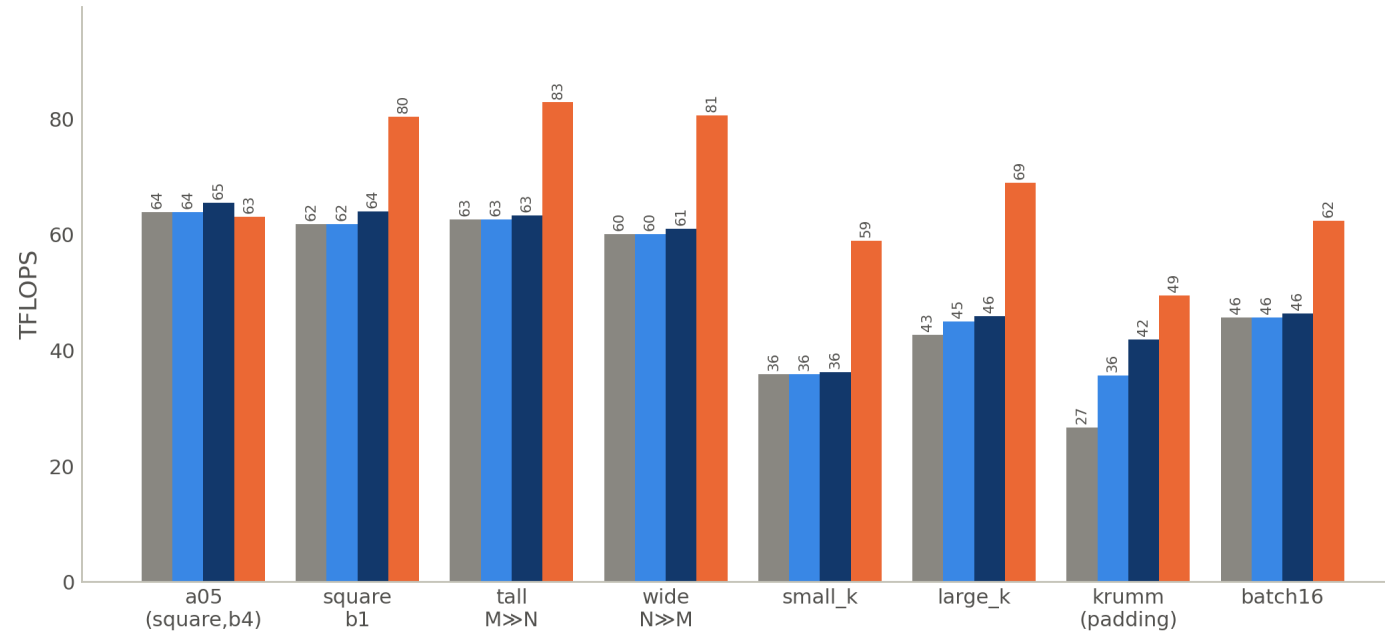
A06 nutzt dieselben acht Regime in Ring-Form (acsp_x, bsp_y → abcy_x)

- ▶ Pro Shape: Tuner vs. Default (8×8) vs. torch.einsum; A06 auch vs. Handkernel.
- ▶ fp16 rein, fp32 Akku; jede Config gegen torch.einsum geprüft (allclose).
- ▶ 8×342 (A05) + 8×171 (A06): 0 Fehlschläge, inkl. Padding-Pfad (krumm).
- ▶ GPU: NVIDIA GB10 — 48 SMs, 25 MB L2, integrierter LPDDR.

Acht Regime decken square / rechteckig / K-Extreme / unteilbar / Batch ab — fair gegen drei Referenzen, alles korrekt.

Der Tuner bestätigt die Handarbeit

A05: Tuner ≈ Default (= die A05-Handarbeit) — Bench-Best bleibt unter cuBLAS



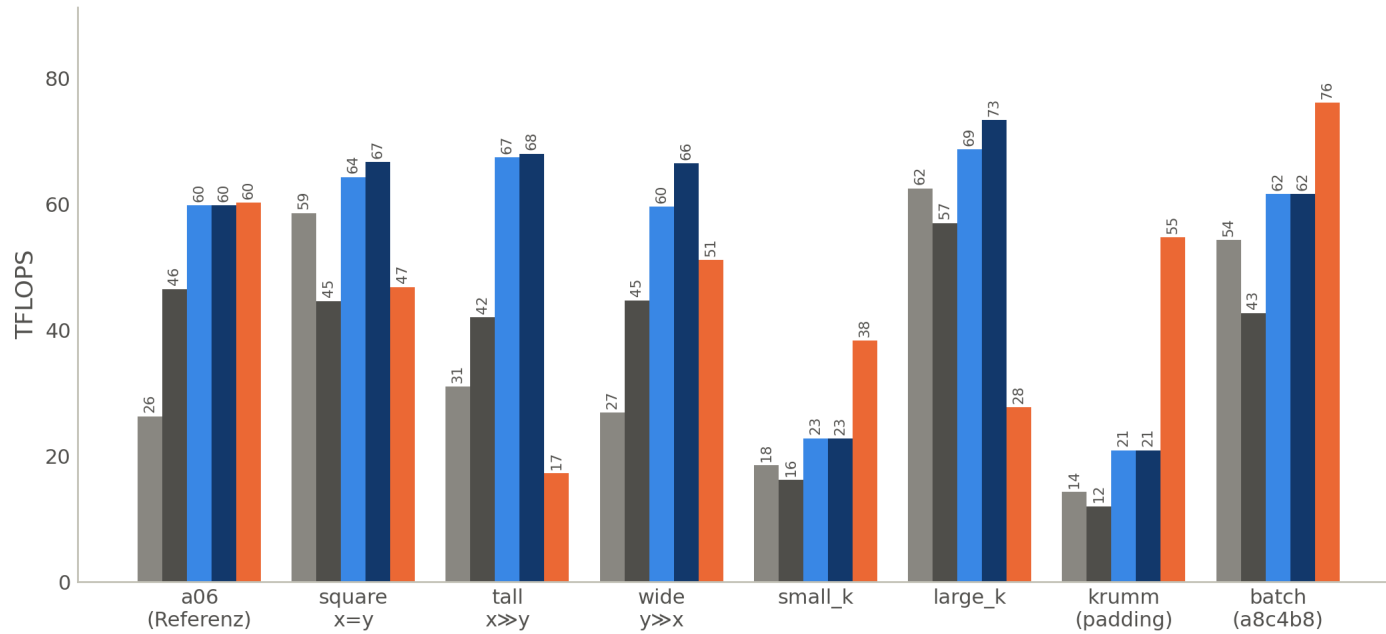
Legend: Default (8x8) · Tuner (top-7) · Bench Best (voller Sweep) · torch.einsum (cuBLAS)
 GB10 · Default (8x8) = die von Hand getunte A05-Config · Bench Best = bestes von 342 · torch = cuBLAS

- ▶ Tuner-Pick (top-7) ≈ Default → bestätigt die Hand.
- ▶ Bench-Best (von 342) nur knapp drüber; krumm +58 %.
- ▶ Aber: selbst die Bench-Best bleibt auf GEMM unter cuBLAS.

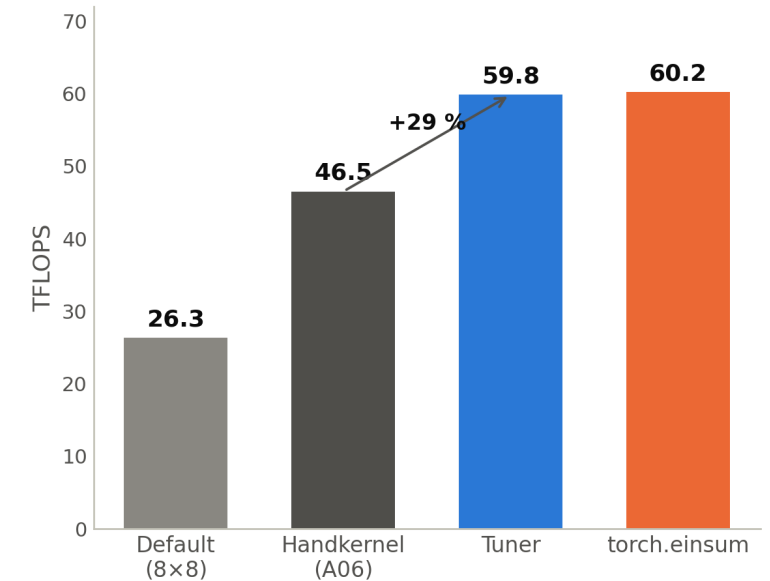
Warum verliert selbst unsere Bench-Best gegen torch? cuBLAS ist eine gereifte GEMM-Library (hand-optimiertes SASS, Split-K, Pipelining) — das schlägt ein einfaches Template.

Transfer gelingt — Tuner schlägt den Handkernel

A06: Tuner » Default (8×8) — und schlägt den Handkernel (Referenz +29 %)



A06-Referenz: der Tuner schlägt den Handkernel und liegt gleichauf mit torch



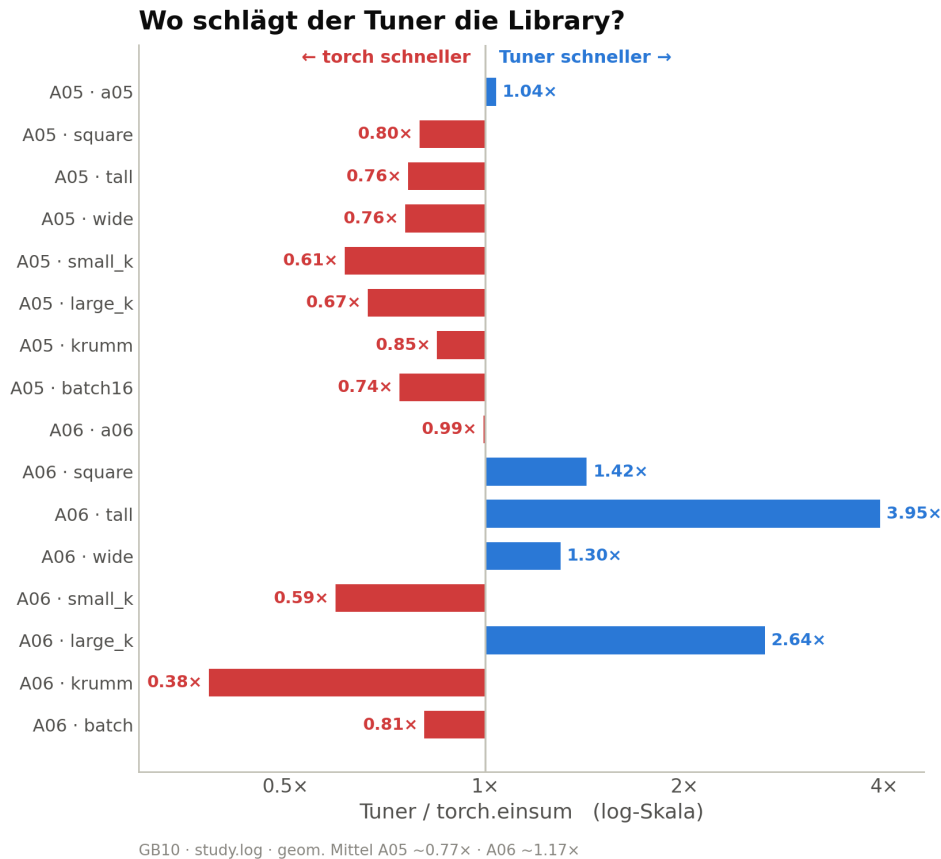
GB10 · study.log · fp16 · acspx,bspy→abcyx (4,3,64,64,1536)×(4,64,64,1152)

■ Default (8×8)
 ■ Handkernel (Ref.-Tiling)
 ■ Tuner (top-7)
 ■ Bench Best (voller Sweep)
 ■ torch.einsum (cuBLAS)

GB10 · Handkernel = festes A06-Referenz-Tiling (2×3) auf jede Shape · Bench Best = bestes von 171 · torch = cuBLAS

Anders als GEMM: unsere Bench-Best schlägt torch, wo dessen Ring-Pfad schlecht ist (tall, large_k); der Tuner schlägt zudem den Handkernel (+29 %, aus k_prim=32).

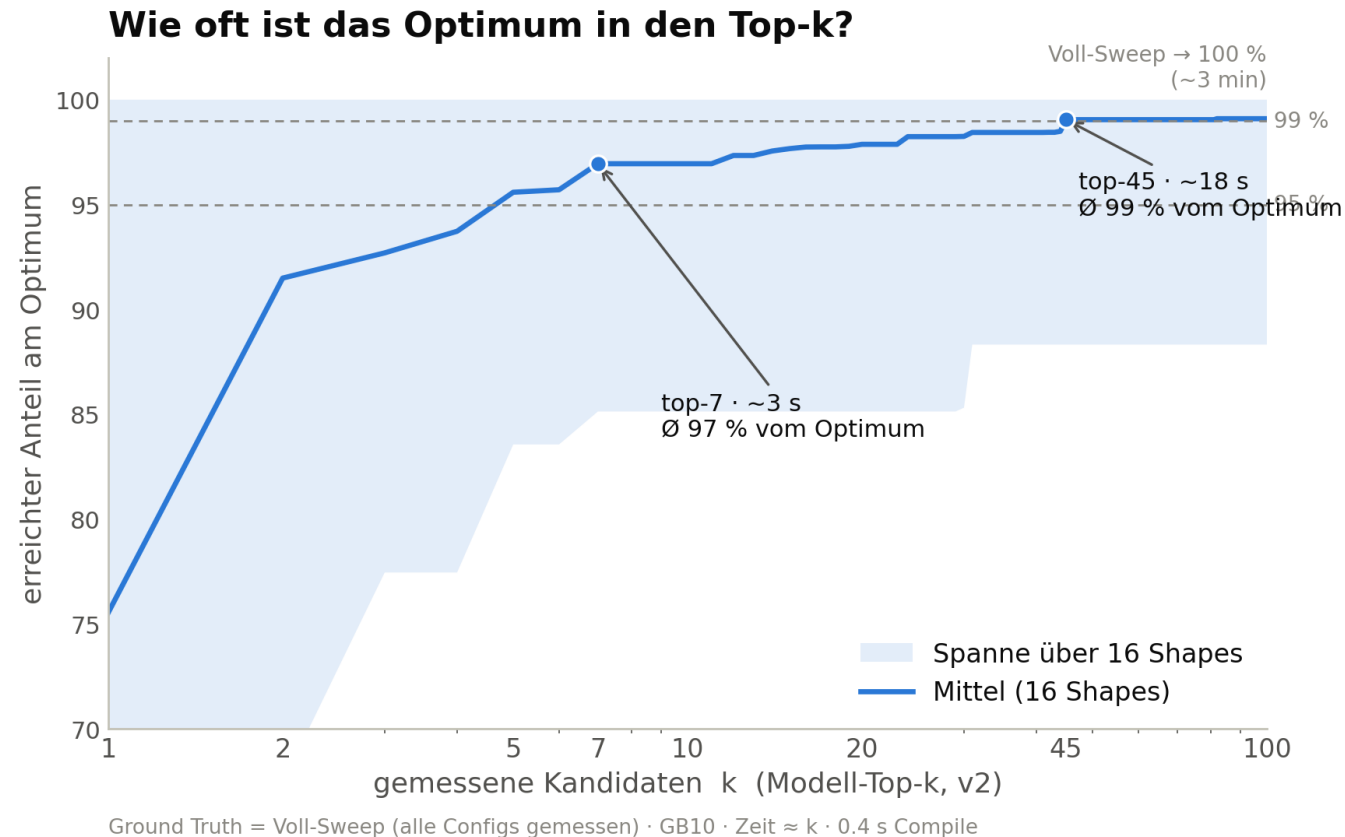
Tuner vs. cuBLAS / torch.einsum



- ▶ GEMM (A05): cuBLAS gewinnt (Tuner ~77 % von torch) — erwartbar.
- ▶ Ring (A06): im Mittel ~1.17x vorn, sehr shape-abhängig.
- ▶ Großer Gewinn, wo torchs Pfad schlecht ist (tall 3.95x, large_k 2.65x).
- ▶ A06_TORCH=16.18 aus dem Assignment ist veraltet → frisch 60.2.

Der Wert ist nicht „schneller als alles“, sondern: ohne Handarbeit über beliebige Kontraktionen brauchbar — und stark, wo die Library keinen guten Pfad hat.

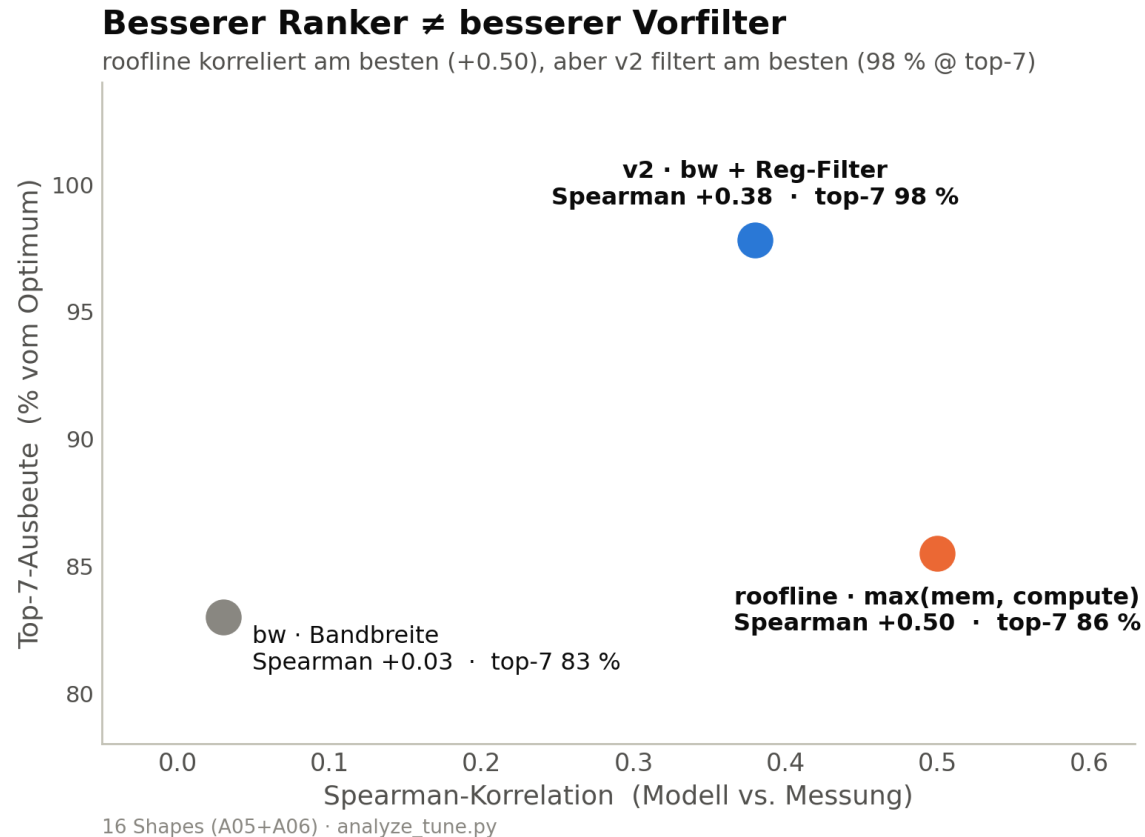
Wie oft ist das Optimum in den Top-k?



- ▶ **top-7 $\cdot \sim 3$ s $\cdot \geq 95\%$**
- ▶ **top-45 $\cdot \sim 18$ s $\cdot \geq 99\%$**
- ▶ **Voll-Sweep $\cdot \sim 3$ min $\cdot 100\%$**
- ▶ Exakt-Beste sitzt tief — Spitzenfeld liegt im Messrauschen.

Als Vorauswahl reicht das Modell fast immer; als exakter Treffer selten — top-7 ist der Sweet Spot.

Besserer Ranker $\neq$ besserer Vorfilter

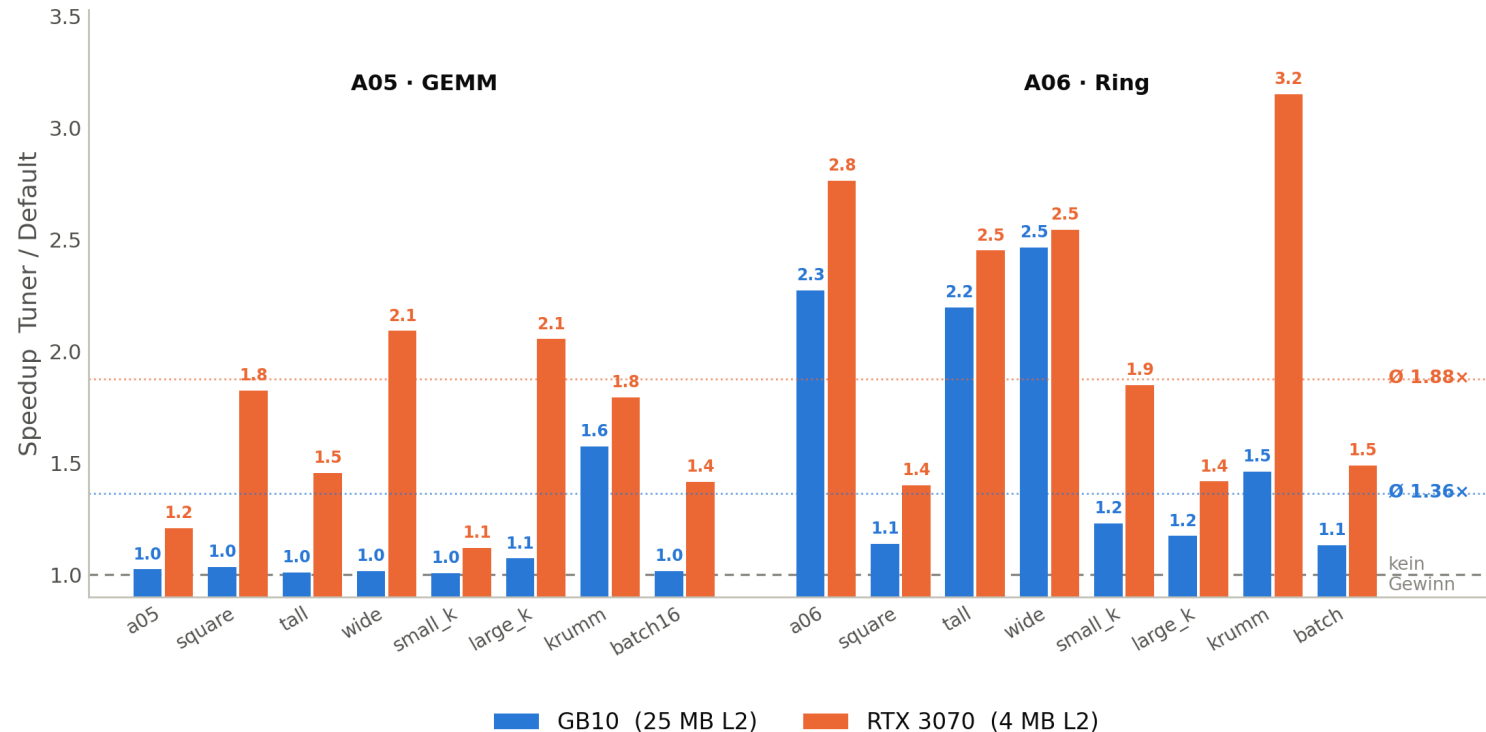


- ▶ bw pur: +0.03 hier — aber die richtige Physik für kleine-L2-GPUs (3070).
- ▶ v2 (bw + Reg-Filter): +0.38 · 97.8 % top-7 → im Tuner verbaut.
- ▶ roofline (L2-bewusst): +0.50 beste Korrelation, nur 85.5 % top-7.

Höchste Korrelation $\neq$ beste Top-k-Ausbeute — für die Praxis zählt der Vorfilter, also v2.

Derselbe Hebel, aber andere Config pro GPU

Der Tuning-Hebel wirkt auf beiden GPUs — auf der 3070 stärker



relativer Speedup (unitless) — absolute TFLOPS beider Karten sind nicht vergleichbar · beste Config in 16/16 Shapes je GPU verschieden

Tuning hilft auf beiden Karten (Ø 1.36× GB10, 1.88× 3070) und die optimale Config ist in 16/16 Shapes verschieden — genau deshalb: GPU-spezifisch tunen + cachén.

Was der Tuner pro GPU wählt — und was es kostet

Optimale Config unterscheidet sich pro GPU (m/n/k-Prim · L2-Gruppe)

Shape	GB10 (25 MB L2)	RTX 3070 (4 MB L2)
<i>A05 · GEMM</i> a05	128/128/64 8×4	64/128/64 8×2
tall	64/256/64 2×4	64/128/64 2×2
small_k	64/256/64 4×2	128/128/32 8×2
krumm	128/128/64 2×2	256/64/32 2×8
<i>A06 · Ring</i> a06	128/128/32 4×2	128/64/64 2×2
tall	128/128/32 2×4	64/128/64 2×2
large_k	128/128/32 8×4	64/128/32 8×2
krumm	64/256/64 8×4	64/64/128 8×4

Muster: GB10 → große 128×128-Tiles (großes L2 verträgt sie) · 3070 → kleineres k_prim=32, oft asymmetrisch/64-breit

- ▶ Optimum in 16/16 Shapes verschieden — pro GPU eine andere Config.
- ▶ Autotuner (v2, top-7) trifft Ø 96 % (GB10) / 90 % (3070) des Optimums.
- ▶ Kosten: GB10 ~3 s/Shape; 3070 (WSL-Compile) ~3 s–2 min — Compile dominiert.
- ▶ Roofline schaltet sein Regime selbst um; praktisch bleibt v2 der beste Vorfilter.

Pro GPU von Hand zu tunen ist nicht machbar — das nimmt der gecachte Tuner ab, ohne auf eine Karte zu overfitten.

Wann sich Tuning lohnt: Cache & Amortisierung

- ▶ Eine Kontraktion läuft ~8 ms; Tuning ~3 s ist ein Einmalposten.
- ▶ Im Netz sind Layer-Dims fix → millionenfach dieselbe Shape.
- ▶ Config-Cache: Key = Einsum + Shapes + GPU-Modell.
- ▶ Genau so machen es cuBLAS, Triton (autotune), torch.compile.

~3 s → 0 s

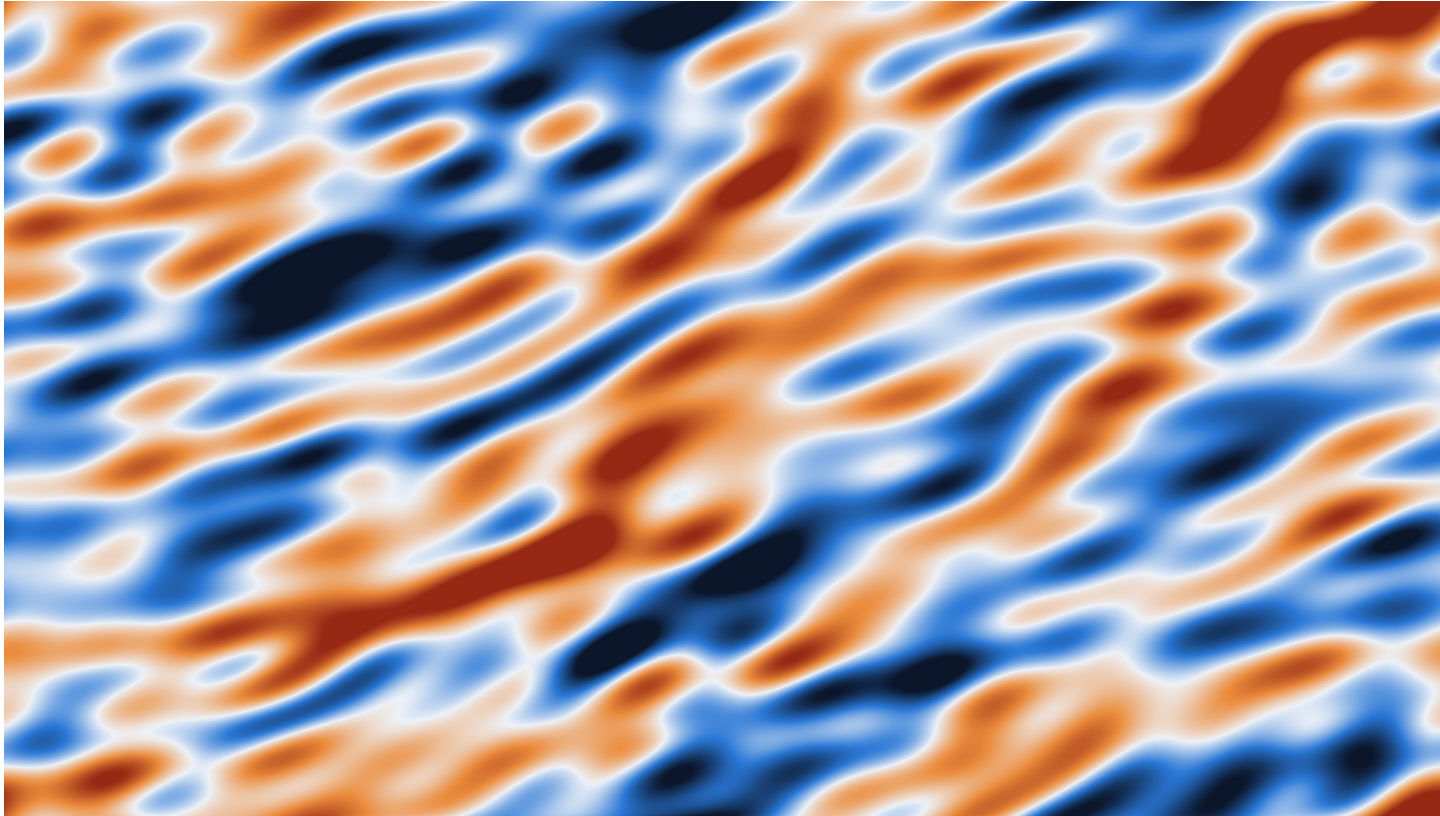
erste getunte Shape → jede weitere aus dem Cache

End-to-end bestätigt

16 Shapes getunt & gecacht, 2. Aufruf Cache-Hit,
Top-7 bei ~95–100 %.

Break-even in Sekunden bei fester Shape; für stark dynamische Shapes hilft Bucketing/Padding.

Der Tuner malt ein Bild — live auf dem DGX



- ▶ Full-HD-Plasma (2 MPixel) = ein großes GEMM: $Y \cdot X$.
- ▶ 8×8-Default vs. Tuner-Config — gleiche Mathe.
- ▶ Gleiches Bild, beide gegen torch geprüft.
- ▶ Krumme Shape → Default padded schlecht → Tuner gewinnt.

▶ `python demo_paint.py`

enumerate 486 → prune → Top-7 messen → schnellere Config, live. Gleiches Bild, aber der Tuner ist schneller da.

Eingrenzen ist sicher — entscheiden muss man messen

- ▶ **A05:** Tuner reproduziert Hand ($\emptyset$ 1.03×), gewinnt bei krummen Shapes (+58 %).
- ▶ **A06:** sauberer Transfer, ein zusätzlicher Kernel-Typ, schlägt den Handkernel (+24 %).
- ▶ **cuBLAS:** auf GEMM nicht geschlagen — Wert ist Allgemeinheit + Gewinne ohne guten Library-Pfad.
- ▶ **Modell:** v2 bester Vorfilter (97.8 % @ top-7), roofline bester Ranker — messen bleibt entscheidend.
- ▶ **Praxis:** GPU-spezifisch + gecacht — auf GB10 und 3070 bestätigt: Hebel wirkt, beste Config je GPU verschieden.